

OPTIMISATION APPROACH FOR MACHINE LEARNING

Gradient of a function

Given a function f , the gradient of the function always points in the direction of steepest ascent.

Let us define x_1, x_2 such that $x_2 = x_1 + \alpha d$ and we want direction d such that $f(x_2) > f(x_1)$

By Taylor's theorem,

$$f(x_2) = f(x_1) + \alpha df'(x_1) + \alpha^2 d^2 f''(x_1) + \dots$$

If we make α small then all terms beyond first order will tend to zero.

$$f(x_2) - f(x_1) > 0$$

$$\alpha df'(x_1) > 0$$

Now in this if we take $d = f'(x_1)$ then $\alpha(f'(x_1))^2 > 0$ for all x_1

Gradient Descent

Idea: Move in the direction of the steepest descent to minimise the function.

given a starting point $\mathbf{x}^{(0)} \in X$

repeat

1. compute $\Delta \mathbf{x}_{\text{gd}}^{(t)}$
2. choose a step size $\alpha > 0$
3. update: $\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} + \alpha \Delta \mathbf{x}_{\text{gd}}^{(t)}$

until stopping criterion is satisfied.

$$\Delta \mathbf{x}_{\text{gd}}^{(t)} = \arg \min_{\|\mathbf{d}\|_2=1} \nabla f(\mathbf{x}^{(t)})^\top \mathbf{d}$$

$$\Delta \mathbf{x}_{\text{gd}}^{(t)} = -\nabla f(\mathbf{x}^{(t)})$$

Steepest descent under the ℓ_2 norm

The Gradient considers all directions simultaneously and chooses the direction of maximum decrease

Coordinate Descent

Idea: Minimise the function by optimising one coordinate at a time.

given a starting point $\mathbf{x}^{(0)} \in X$

repeat

1. compute $\Delta \mathbf{x}_{\text{cd}}^{(t)}$
2. choose a step size $\alpha > 0$
3. update: $\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} + \alpha \Delta \mathbf{x}_{\text{cd}}^{(t)}$

until stopping criterion is satisfied.

$$\Delta \mathbf{x}_{\text{cd}}^{(t)} = \arg \min_{\|\mathbf{d}\|_1=1} \nabla f(\mathbf{x}^{(t)})^\top \mathbf{d}$$

$$= -\text{sign}(\nabla f(\mathbf{x})_i) \mathbf{e}_i$$

$$i = \arg \max_j |\nabla f(\mathbf{x})_j|$$

Steepest descent under the ℓ_1 norm

At each step, choose the coordinate with the largest gradient magnitude and move in its descent direction

Difference between the two algorithms

Gradient Descent	Coordinate Descent
optimises the function at each step overall for all directions	optimises the function sequentially by updating only one direction at a time
works well when function is not separable or coupled	works well when function is separable, since it moves only one coordinate in a iteration

Full Batch Gradient Descent

Idea: Use the entire training dataset to calculate the gradient before making one update.

given a starting point $\mathbf{x}^{(0)} \in X$

repeat

1. compute $\Delta \mathbf{x}^{(t)} = -\nabla f(\mathbf{x}^{(t)})$
2. choose a step size $\alpha > 0$
3. update: $\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} + \alpha \Delta \mathbf{x}^{(t)}$

until stopping criterion is satisfied.

For a dataset with n training examples,

$$\nabla f(\mathbf{x}^{(t)}) = \frac{1}{n} \sum_{i=1}^n \nabla L_i(\mathbf{x}^{(t)})$$

The Gradient is calculated using all training examples before every update

Stochastic Gradient Descent

Idea: Instead of using the entire dataset, use one randomly selected training example to calculate the gradient.

given a starting point $\mathbf{x}^{(0)} \in X$

repeat

1. randomly choose a training example i
2. compute $\Delta \mathbf{x}^{(t)} = -\nabla L_i(\mathbf{x}^{(t)})$
3. choose a step size $\alpha > 0$
4. update: $\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} + \alpha \Delta \mathbf{x}^{(t)}$

until stopping criterion is satisfied.

Equivalently,

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \alpha \nabla L_i(\mathbf{x}^{(t)})$$

The Gradient is calculated using one randomly selected training example at each update

Mini Batch Gradient Descent

Idea: Instead of using the entire dataset, use one randomly selected training example to calculate the gradient.

given a starting point $\mathbf{x}^{(0)} \in X$

repeat

1. randomly choose a mini-batch B of m training examples
2. compute $\Delta \mathbf{x}^{(t)} = -\frac{1}{m} \sum_{i \in B} \nabla L_i(\mathbf{x}^{(t)})$
3. choose a step size $\alpha > 0$
4. update: $\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} + \alpha \Delta \mathbf{x}^{(t)}$

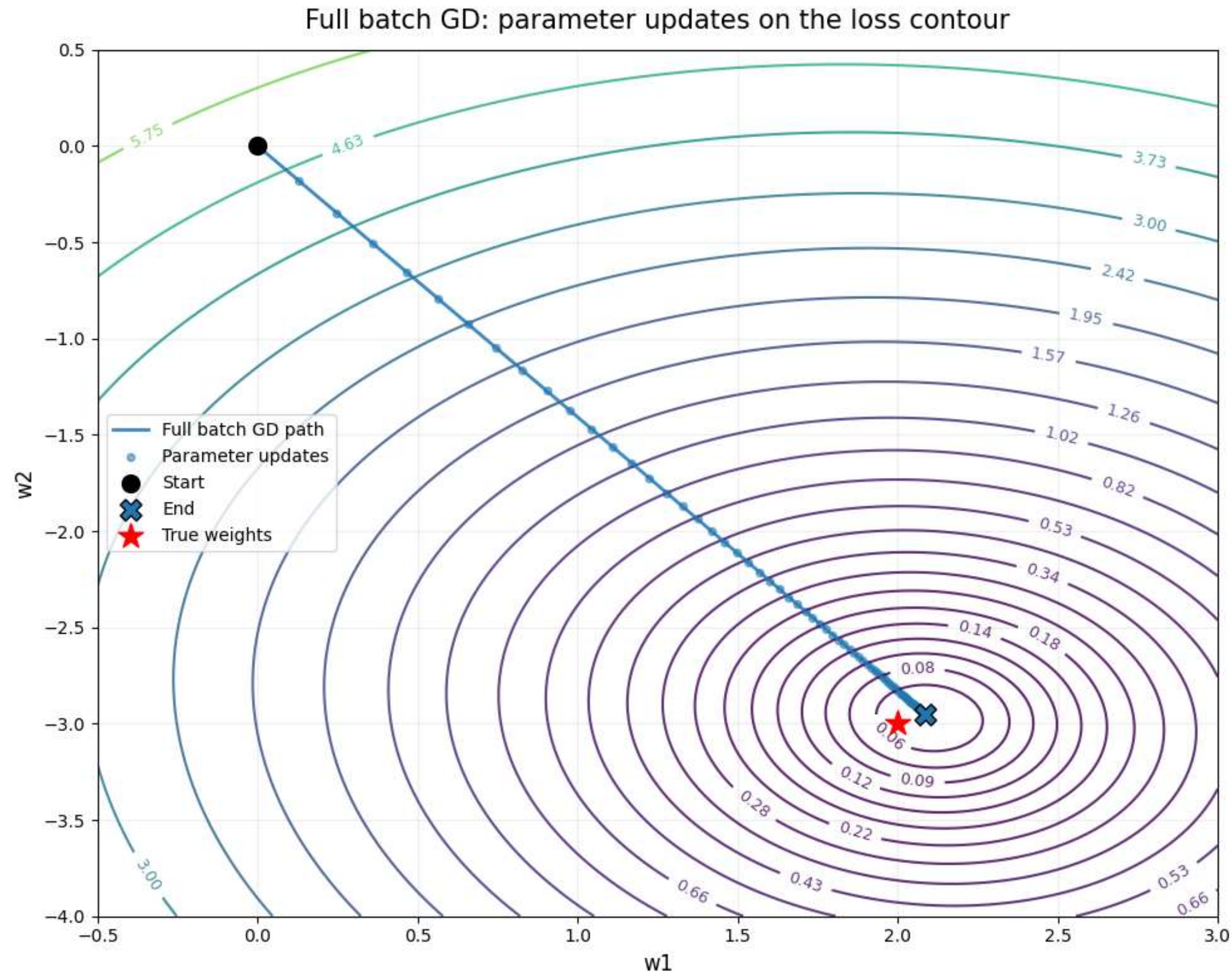
until stopping criterion is satisfied.

Equivalently,

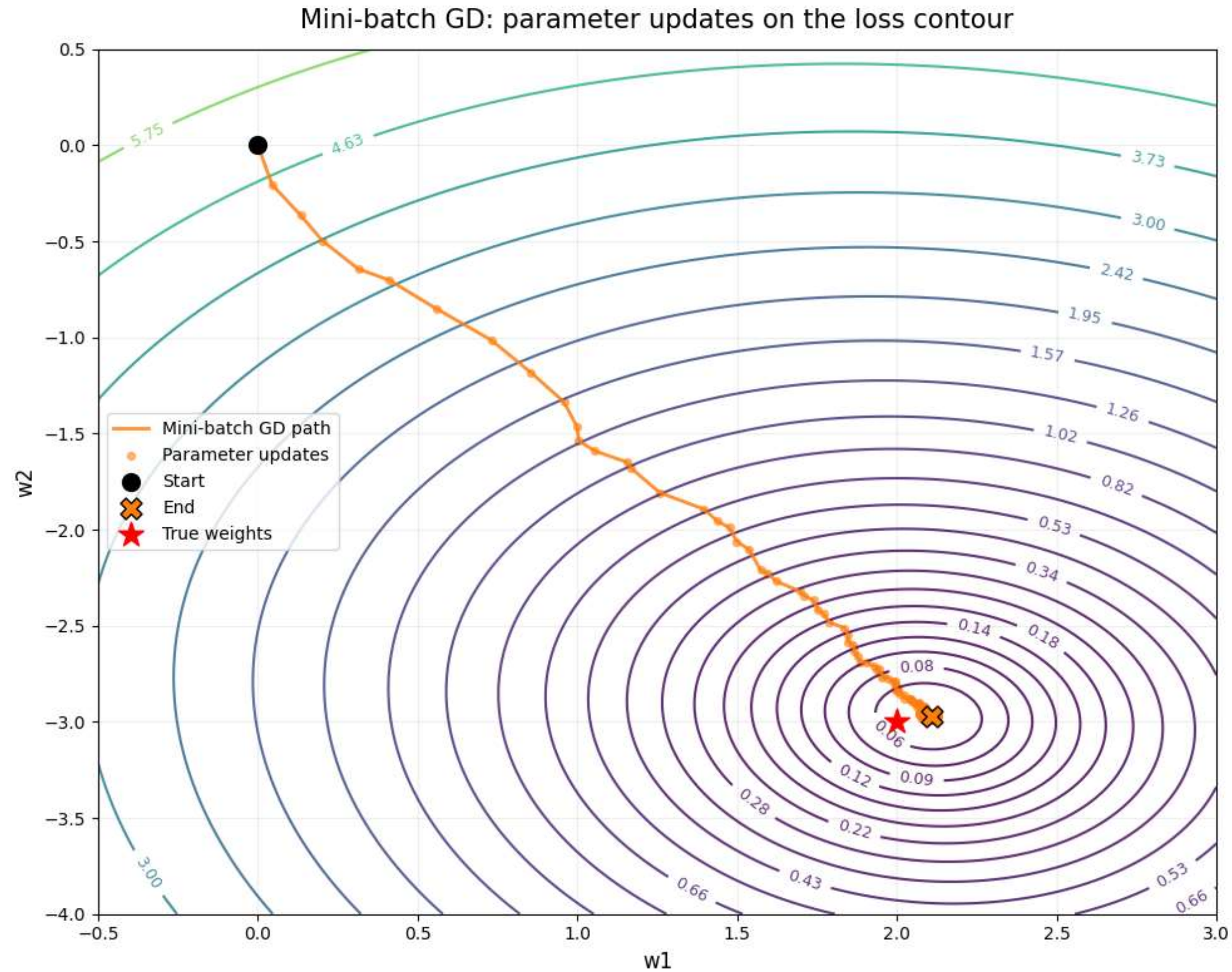
$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \alpha \frac{1}{m} \sum_{i \in B} \nabla L_i(\mathbf{x}^{(t)})$$

The Gradient is calculated using a small subset of the training examples at each update

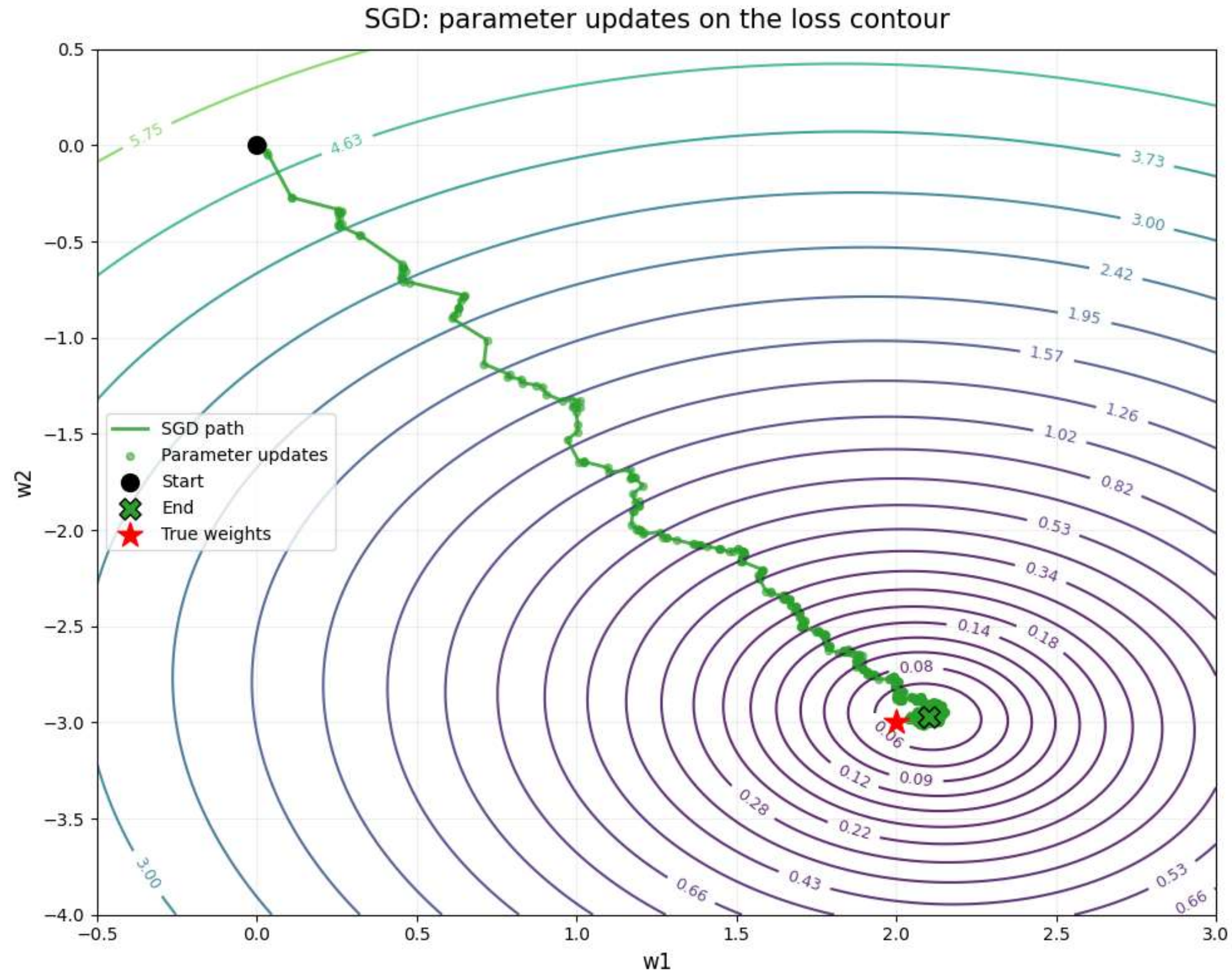
Weight Updates in Full Batch Gradient Descent



Weight Updates in Mini Batch Gradient Descent



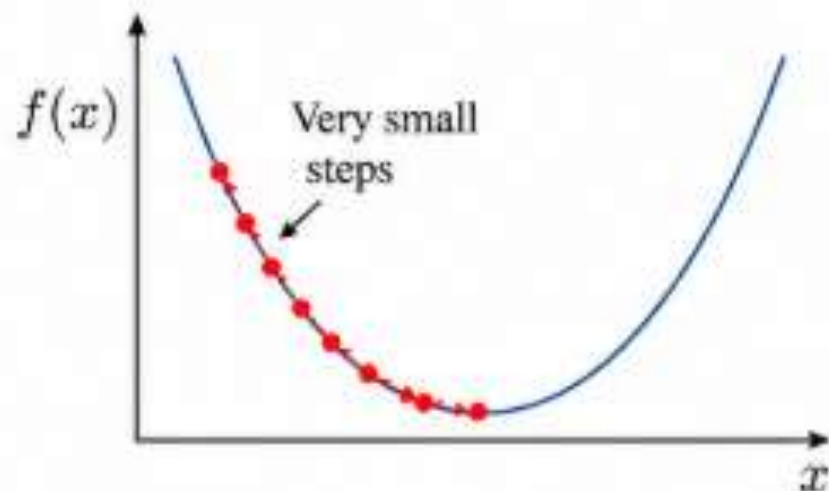
Weight Updates in SGD



Effect of Learning Rate

Idea: The learning rate, α controls the step size taken towards the minimum at each iteration.

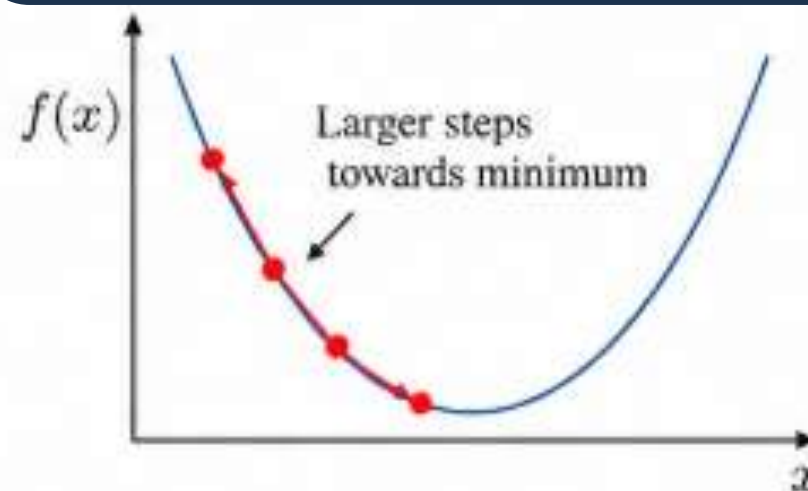
Learning Rate Too Small
(α too small)



- Takes very small steps
- Converges slowly
- Requires many iterations

Slow convergence

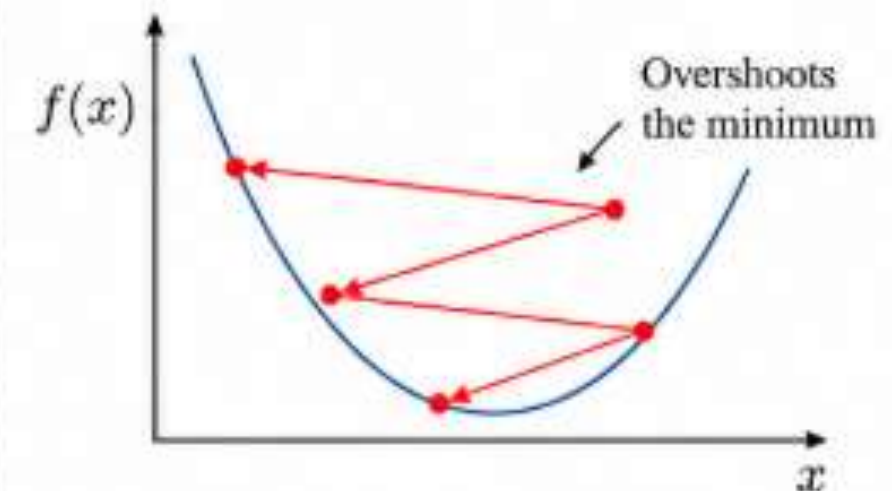
Appropriate Learning
Rate
(α is good)



- Takes sufficiently large steps
- Converges quickly and smoothly
- Reaches the minimum efficiently

Converges efficiently

Learning Rate Too Large
(α too large)



- Takes very large steps
- Overshoots the minimum
- May oscillate or diverge

Oscillates/ Diverges

THANK YOU